SIMATS ENGINEERING

CO2_AT3 Error Analysis Task

COURSE NAME: Software Testing COURSE CODE: CSA3712

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 20%

Sl. No	Questions	Bloom's Level
1	A tester designs test cases for an online shopping system using equivalence partitioning but incorrectly groups input ranges (e.g., valid and invalid inputs mixed in one class). Additionally, boundary values are not considered. Analyze the given test design and identify all errors in equivalence class partitioning and boundary value analysis. Examine the root causes of these mistakes with proper reasoning. Propose corrected test cases with proper classification and boundary coverage. Justify how your corrections improve defect detection effectiveness.	3
2	A QA engineer performs random testing on a banking application and claims high coverage without using any structured techniques. Critical defects in transaction limits and boundary conditions are missed. Evaluate the testing approach and identify errors in strategy and coverage assumptions. Analyze the root causes behind missing critical defects. Suggest appropriate black box testing techniques to correct the approach. Justify how these techniques ensure better test adequacy and reliability.	3
3	A developer performs white box testing but creates an incomplete control flow graph, missing decision nodes and loops, resulting in poor path coverage. Analyze the errors in the control flow graph and path testing approach. Identify the root causes for inadequate test coverage. Propose corrections by constructing a proper control flow representation and identifying all independent paths. Justify how improved path testing ensures better coverage and defect detection.	5

4	A testing team reports 100% statement coverage for a module but fails to detect logical errors in conditional statements. Evaluate the adequacy of the testing performed and identify gaps in coverage criteria. Analyze why statement coverage alone was insufficient. Recommend improved white box testing strategies such as branch or path coverage. Justify how the revised approach enhances overall test effectiveness and error detection.	5
5	A tester uses Equivalence Class Partitioning for an age-validation field (0-100) but defines only two classes: Negative numbers and Positive numbers. Analyze the errors in this partitioning strategy. Identify why this fails to account for valid vs. invalid age limits and the absence of Boundary Value Analysis. Propose a corrected set of partitions and justify how they ensure more robust input validation.	3
6	A developer claims full Condition Coverage for a complex `if(A && B	
7	A QA team applies Decision Table Testing for a discount system but misses several combinations of "Premium Member" and "Coupon Applied," leading to price calculation errors. Analyze the errors in the construction of the decision table. Identify why the current approach is incomplete. Propose a revised, exhaustive decision table and justify its effectiveness in preventing production defects.	3
8	During unit testing of a looping structure, a tester checks only the first and last iterations (n=1 and n=max). Defects occurring at n=0 and n=max+1 are missed. Evaluate the errors in this Loop Testing strategy. Analyze the root causes behind these omissions. Recommend a corrected approach for testing simple and nested loops, and justify how it ensures path integrity.	5
9	An engineer utilizes Error Guessing as the primary testing technique for a safety-critical braking system, ignoring structured black box methods. Identify the errors in relying solely on ad-hoc techniques for high-risk modules. Analyze the impact on test adequacy. Suggest a combination of BVA and State Transition Testing to fix the strategy, justifying the increased safety assurance.	5

10	A tester designs a Control Flow Graph (CFG) for a module but fails to represent the "else" branches of multiple nested if statements, leading to 100% statement coverage but 0% branch coverage on error paths. Analyze the errors in graph construction. Identify the root causes for the missing nodes. Propose a corrected CFG and independent paths to ensure branch adequacy.	5

1. Online Shopping – Equivalence Partitioning and Boundary Value Analysis

Errors identified: Valid and invalid inputs are grouped together; minimum and maximum limits are not tested; test cases are not selected systematically.

Root causes: The tester may not have clearly identified the input rules and valid ranges. Without boundary testing, defects near limits can remain undetected.

Corrected test cases: Assume shopping quantity must be between 1 and 10.

Class	Example input	Expected result
Invalid: below minimum	0	Reject
Valid: within range	5	Accept
Invalid: above maximum	11	Reject
Boundary: minimum	1	Accept
Boundary: maximum	10	Accept
Just below minimum	0	Reject
Just above maximum	11	Reject

Justification: Separating valid and invalid classes improves test selection. Boundary Value Analysis helps detect off-by-one errors and incorrect range checks.

2. Banking Application – Random Testing

Errors identified: Random inputs are used without a planned strategy; transaction limits and boundary conditions are missed; high coverage is claimed without evidence.

Root causes: Random testing may miss important edge cases, and repeated inputs may test the same behavior.

Test adequacy cannot be judged only by the number of random tests.

Technique	Example test
Equivalence Partitioning	Test valid and invalid transaction amounts.
Boundary Value Analysis	Test minimum, maximum, and values just outside limits.
Decision Table Testing	Test combinations of account types, balances, and transaction limits.
Negative Testing	Test zero, negative, or incorrectly formatted amounts.

Justification: Structured black-box techniques cover valid, invalid, and boundary inputs systematically. Random testing can supplement these techniques but should not replace them.

3. White-Box Testing – Incomplete Control Flow Graph

Errors identified: Decision nodes are missing; loops and their return edges are not represented; independent paths are not identified correctly.

Root causes: The developer may have focused only on statements and ignored the branching and looping structure.

Start → Read x → x > 0? \n True → Print Positive → End\n False → Print Non-positive → End

Path	Execution
P1	Start → Read x → Decision True → Print Positive → End
P2	Start → Read x → Decision False → Print Non-positive → End

For a loop, the graph must also include the loop condition, loop body, and return edge.

Justification: A complete CFG identifies decision outcomes and loop behavior. Testing independent paths improves path coverage and helps detect logic errors.

4. Statement Coverage – Logical Errors in Conditions

Errors identified: The team reports 100% statement coverage but does not test every decision outcome. Conditions may evaluate only one way, leaving logical errors undetected.

Root causes: Statement coverage checks whether each statement executes at least once. It does not guarantee that both True and False outcomes are tested.

Example: if (age >= 18)

Test case	Age	Expected outcome
TC1	20	True – eligible
TC2	16	False – not eligible
TC3	18	True – boundary value
TC4	17	False – boundary value

Justification: Branch Coverage tests both outcomes of decisions. Condition Coverage checks individual Boolean conditions, while Path Coverage examines execution paths. Using suitable criteria improves defect detection beyond statement coverage alone.

5. Age Validation – Incorrect Equivalence Classes

Errors identified: Only negative and positive numbers are considered; zero and the maximum valid age are not handled properly; values above 100 are not separated into an invalid class; boundaries are ignored.

Assumption: Valid ages are integers from 0 to 100, inclusive.

Class	Input	Expected result
Invalid: below range	-1	Reject
Valid: lower boundary	0	Accept
Valid: middle value	50	Accept
Valid: upper boundary	100	Accept
Invalid: above range	101	Reject
Invalid: non-integer	20.5	Reject if only integers are allowed
Invalid: non-numeric	abc	Reject

Boundary Value Analysis: Test -1, 0, 1, 99, 100, and 101.

Justification: Correct partitions cover valid and invalid inputs, while boundary testing detects errors at the minimum and maximum limits.

6. Condition Coverage – Complex if (A && B)

Note: The uploaded question is cut off after "if (A && B". The answer below is based only on the visible topic of Condition Coverage.

Errors identified: Full Condition Coverage may be claimed without testing both True and False outcomes for each condition. Testing only cases where the entire expression is True is insufficient.

Test case	A	B	Overall result
TC1	True	True	True
TC2	False	True	False
TC3	True	False	False
TC4	False	False	False

Justification: These tests demonstrate both outcomes for each condition and cover all combinations of two Boolean conditions. They help identify errors in compound logical expressions.

7. Discount System – Decision Table Testing

Errors identified: Some combinations of Premium Member and Coupon Applied are missing; discount rules may be unclear; incorrect price calculations may go unnoticed.

Assumption: Premium members receive 10% discount and a valid coupon provides an additional 5% discount. The discounts are additive for this example.

Rule	Premium member?	Coupon applied?	Expected discount
R1	Yes	Yes	15%
R2	Yes	No	10%

R3	No	Yes	5%
R4	No	No	0%

Justification: The table covers all four combinations of the two Boolean conditions. Testing each rule helps identify missing cases and incorrect discount calculations. Actual discount rules should follow system requirements.

8. Loop Testing – Missing Boundary Iterations

Errors identified: Only the first iteration (n = 1) and maximum iteration are tested; zero iterations and one iteration beyond the maximum are ignored.

Root causes: The tester has not applied systematic loop testing and has overlooked cases where the loop should not execute or exceeds its allowed limit.

Test case	Number of iterations	Expected behavior
TC1	0	Loop body does not execute
TC2	1	Executes once
TC3	2	Executes twice
TC4	max − 1	Executes within limit
TC5	max	Executes at the limit
TC6	max + 1	Reject or handle according to requirements

For nested loops: Test the inner loop at zero, one, and typical iterations while varying the outer loop. Also test minimum, maximum, and invalid loop limits.

Justification: Systematic loop testing checks initialization, termination, iteration counts, and interactions between nested loops. This improves loop reliability and helps identify boundary defects.

9. Safety-Critical Braking System – Error Guessing

Errors identified: Error Guessing is used as the only technique; testing is not based on structured requirements and risk analysis; important braking limits and state transitions may be missed.

Root causes: Error Guessing depends heavily on tester experience and may miss unexpected defects. For a safety-critical system, ad-hoc testing alone does not provide adequate assurance.

Technique	Example test
Boundary Value Analysis	Test braking pressure and speed at minimum, maximum, and just outside valid limits.
State Transition Testing	Test transitions between normal braking, emergency braking, and fault states.

Error Guessing	Check likely failures, such as sensor errors or delayed braking.
Negative Testing	Test invalid sensor readings and unexpected inputs.

Justification: Combining structured testing with Error Guessing improves coverage of boundary conditions and operating states. This helps identify dangerous failures and strengthens safety assurance.

10. Control Flow Graph – Missing else Branches

Errors identified: The CFG omits else branches from nested if statements; error-handling paths are not represented; statement coverage is incorrectly treated as sufficient despite missing branch coverage.

Root causes: The tester may have mapped only statements executed in successful scenarios and failed to represent all decision outcomes.

Start → Check A? \n A = False → Error A → End\n A = True → Check B? \n B = True → Success → End\n B = False → Error B → End

Path	Execution
P1	Start → A True → B True → Success → End
P2	Start → A True → B False → Error B → End
P3	Start → A False → Error A → End

The corrected CFG includes both outcomes of each decision and the relevant error paths. Testing all three paths provides branch coverage for this example and helps detect defects hidden by successful executions alone.

Conclusion

Error analysis helps identify weaknesses in test design, understand their root causes, and apply suitable black-box and white-box techniques. Equivalence Partitioning, Boundary Value Analysis, Decision Table Testing, Condition Coverage, Branch Coverage, Path Testing, and Loop Testing improve defect detection when selected according to system requirements.

Code of Conduct:

I __B.Satya Hyndavi(192311178)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:B.Satya Hyndavi

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand